„Systemnahe Programmierung“ (BA-INF 034) Wintersemester 2026/2027

Dr. Matthias Frank, Dr. Matthias Wübbeling

Institut für Informatik 4

Universität Bonn

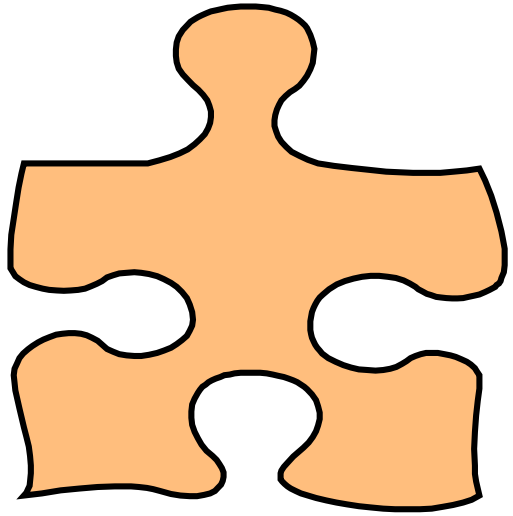
E-Mail: *{matthew, matthias.wuebbeling}@cs.uni-bonn.de*

Sprechstunde: nach der Vorlesung bzw. nach Vereinbarung

UNIVERSITÄT BONN

2. Fortgeschrittene Konzepte der Systemprogrammierung

2. Betriebssysteme/Threads
„Kommunikation innerhalb von Prozessen bzw. zwischen Prozessen eines Rechners“



Teil 1

2.1. Processes

2.2. Threads

2.3. Interprocess Communication IPC

© 2026/2027 Dr. Matthias Frank, Dr. Matthias Wübbeling, Informatik 4

Danksagung - Acknowledgement

- **WS 2008/2009 erste Durchführung**
  dieser Vorlesung im Bachelor-Studiengang
  mit zwei Dozenten.

- **WS 2014/2015 erste Durchführung**
  Vorlesung mit Go

- Im Wesentlichen werden die theoretischen Grundlagen von Prof. Marrón und Mitarbeitern in unserem **Kapitel 2** weitergeführt.

- Die **Folien** waren und bleiben **in Englisch**,
  da sie sich eng an der vorhandenen (klassischen) englisch-sprachigen Literatur orientieren.

Die Dozenten



Prof. Dr. Pedro José Marrón

Institut für Informatik IV
Römerstr. 164
Raum A414

E-Mail: pjmarron@cs.uni-bonn.de
Tel.: 73-4220

Forschung & Lehre:

- Sensor Networks
- Pervasive Computing
- Verteilte Systeme



Dr. Matthias Frank

Institut für Informatik IV
Römerstr. 164
Raum N104b

E-Mail: matthew@cs.uni-bonn.de
Tel.: 73-4550

Forschung & Lehre:

- Systemnahe Programmierung
- Prakt./PG im Bereich systemn. Programmierung
- Kommunikationssysteme
- Mobilkommunikation

Copyright © 2008 Prof. Dr. P. Marrón, Dr. M. Frank, Institut für Informatik IV, Universität Bonn

universitätbonn

Rheinische Friedrich-Wilhelms-Universität Bonn
Institute for Computer Science IV
Sensor Networks and Pervasive Computing Group
Römerstraße 164
D-53117 Bonn

Systemnahe Programmierung
Chapter 2: Processes

Winter Term 2008/2009

Prof. Dr. Pedro José Marrón

UNIVERSITÄT BONN

2.1. Processes - Literature

- [Tanenbaum07] Andrew S. Tanenbaum: **Modern Operating Systems** (International Ed. of 3rd Revsied Edition), *Prentice Hall International / Pearson International Edition*, 2013

„Moderne Betriebssysteme“
3. Auflage deutschsprachig, April 2009

- [Silberschatz08] Abraham Silberschatz, Peter Baer Galvin, Greg Gagne: **Operating System Concepts** (8th Edition), *Wiley & Sons*, 2008

- [Mitchell01] Mark Mitchell, Jeffrey Oldham, Alex Samuel: **Advanced Linux Programming** (1st Edition), *New Riders Publishing*, June 2001,

9. Auflage, International Student Version
Mai 2013

Outline

- **2.1.1. Overview**
  - Motivation
  - Concept
  - Implementation
  - Scheduling
  - Basic operations
- **2.1.2. Utilization**
  - Examples (Linux)
- **2.1.3. Summary**

© 2026/2027 Dr. Matthias Frank, Dr. Matthias Wübbeling, Informatik 4

Outline

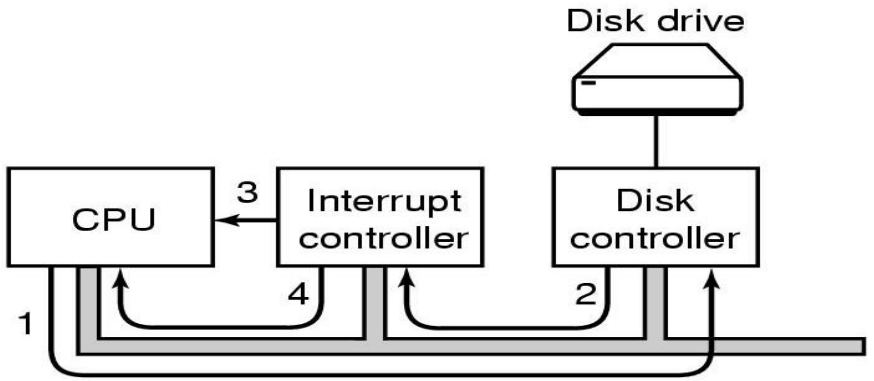
- **2.1.1. Overview**
  - Motivation
  - Concept
  - Implementation
  - Scheduling
  - Basic operations
- 2.1.2. Utilization
  - Examples (Linux)
- 2.1.3. Summary

2.1.1. Overview – High-level Processor Review

- Sequential execution of one program
  - Fetches instruction at program counter (PC)
  - location, executes instruction and increases PC

- Interrupt-driven hardware handling
  - I/O controllers signal events via interrupts
  - changes PC location to specific value

[Tanenbaum01]



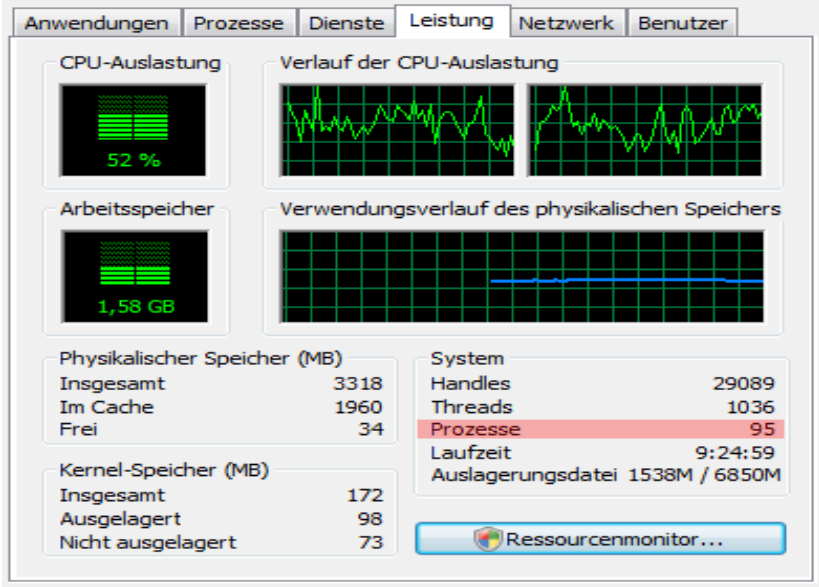
=> CPU only knows one program but **user wants to execute several** in parallel

- Note: current hardware is more complicated
  - Pipelines, hyper-threading, DMA, etc. (c.f. lecture on computer architecture)
  - Different execution modes
    - Kernel mode: full instruction set
    - User mode: limited instruction set (e.g. no I/O instructions, memory protection)

Technische Informatik

The Process Concept

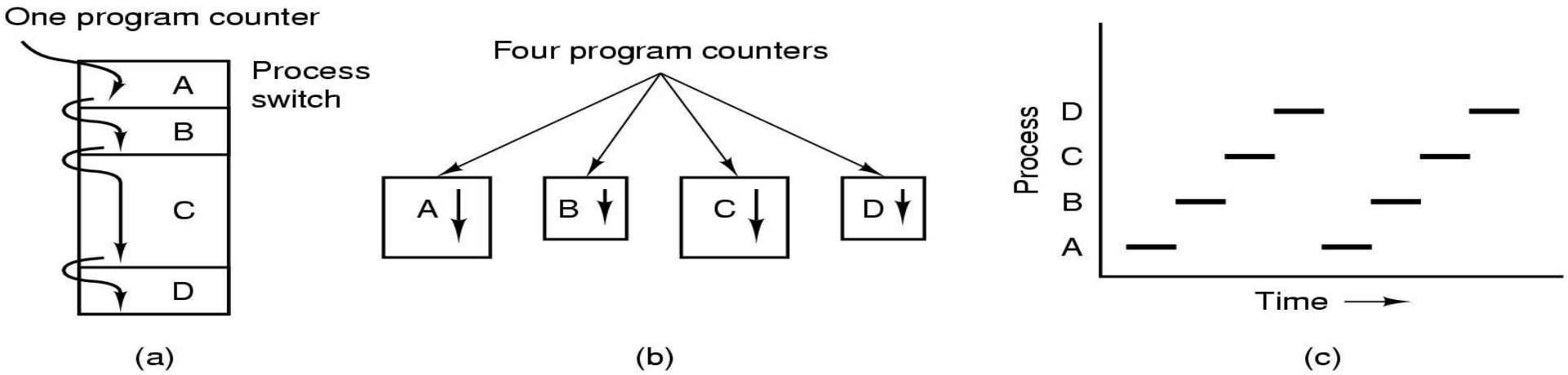
- **A process is an abstraction**
  - Provided by the operating system
  - For a **running instance of a program**
  - Enabling the “simultaneous“ execution of multiple sequentially running instances
- Conceptually, each process
  - Is executed sequentially
  - Has its own virtual processor
  - Runs independent from other processes
- Remarks
  - Historically, differentiation between job (batch) and tasks (interactive)
  - Typically less processors than processes
  - OS switches processes on processors (fast)



2 processors
95 processes

The Process Model

[Tanenbaum01]



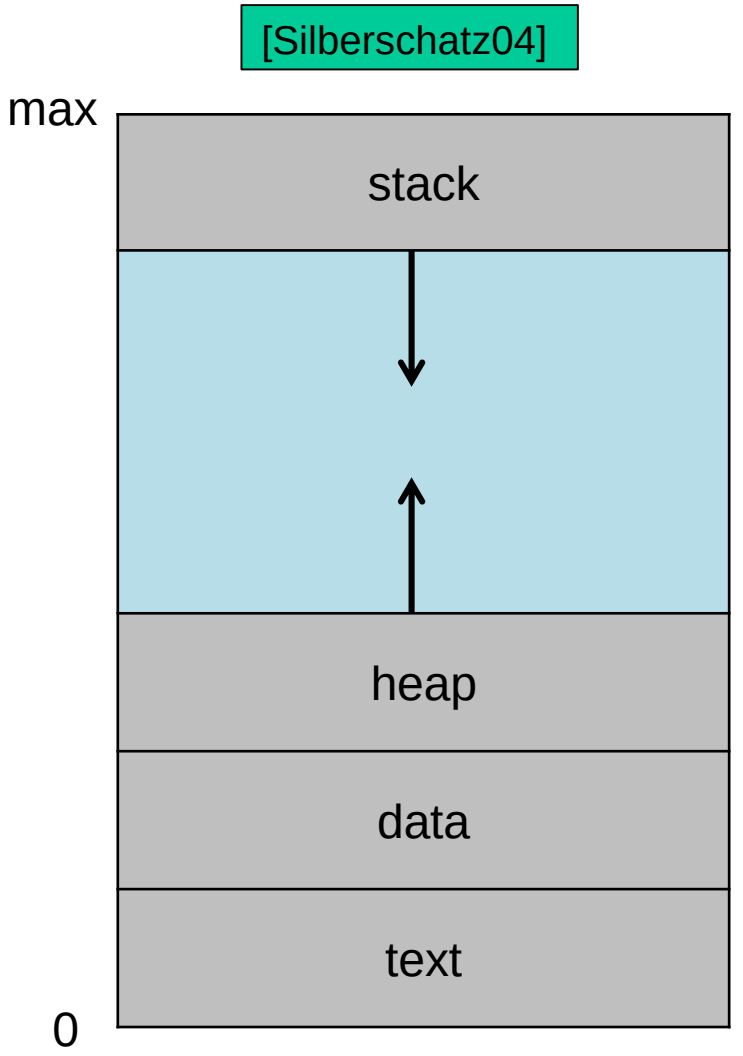
- Multiprogramming of four programs
- Conceptual model of 4 independent, sequential processes **=> fig. (b)**
- Only one program active at any instant **=> fig. (a) + (c)**
- Fast switching by operating system creates illusion of parallelism

Process in Memory

- A process in memory is more than program code

- The memory is limited and consists of sections for
  - Text (program code)
  - Data (global variables)
  - Stack (frames for unfinished procedure calls)
    - Parameters
    - Local variables
    - Return address
  - Heap (dynamically allocated data)

- Processes can only access their assigned memory
  - Processor runs in user mode
  - Memory segregation

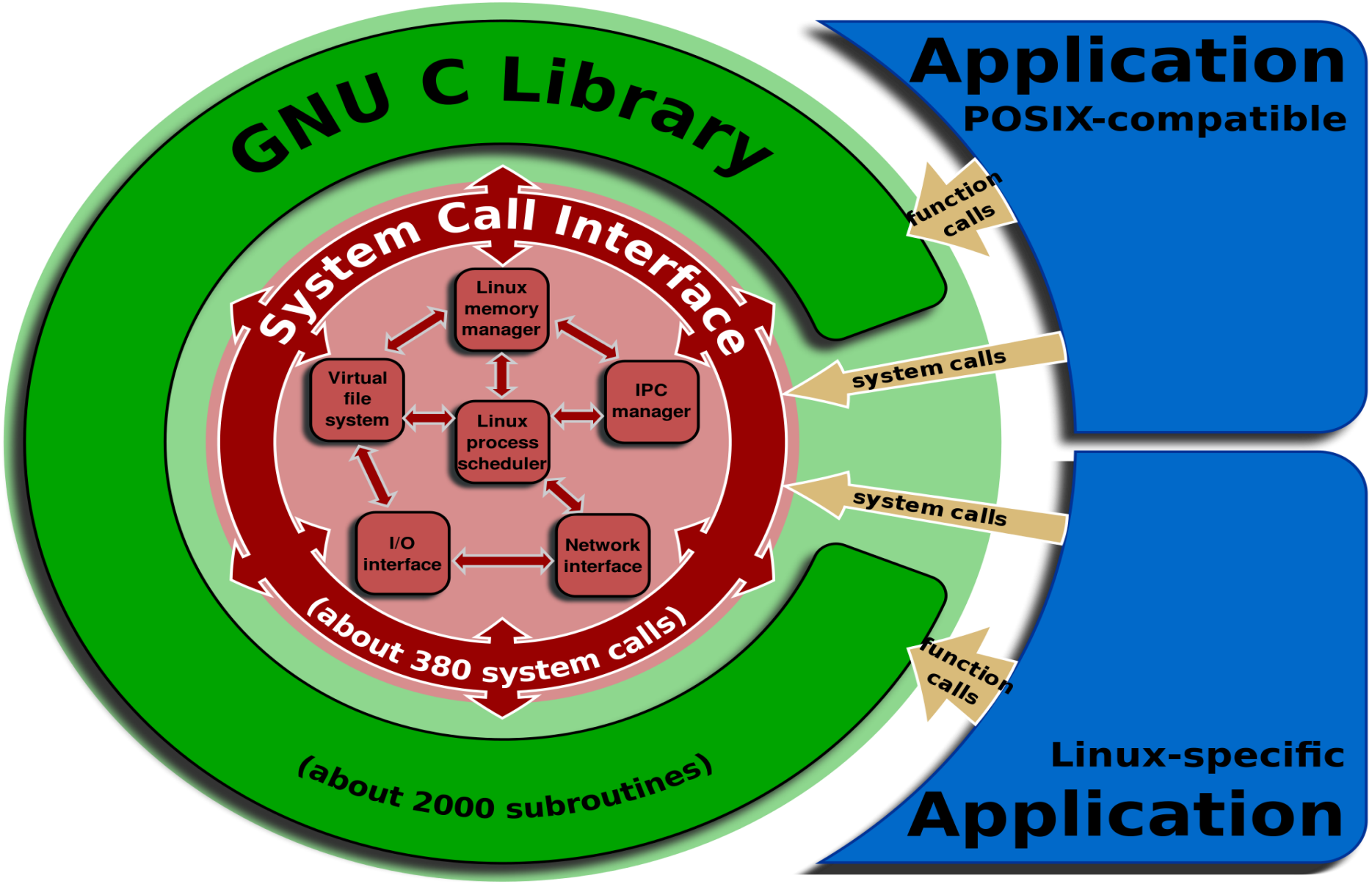


System Calls

- Most operating systems (e.g. Linux and Windows) distinguish between **kernel-mode** and **user-mode**
- Kernel-mode processes (privileged) can access everything
  - Drivers
  - Kernel modules
- User-mode processes are only allowed to read their own memory
- **System-Calls** are the interface to the OS
  - Reading / writing Files, Sockets, etc.
  - Inter-Process Communication (IPC)
  - read from the C standard library `sys_read` internally



System Calls

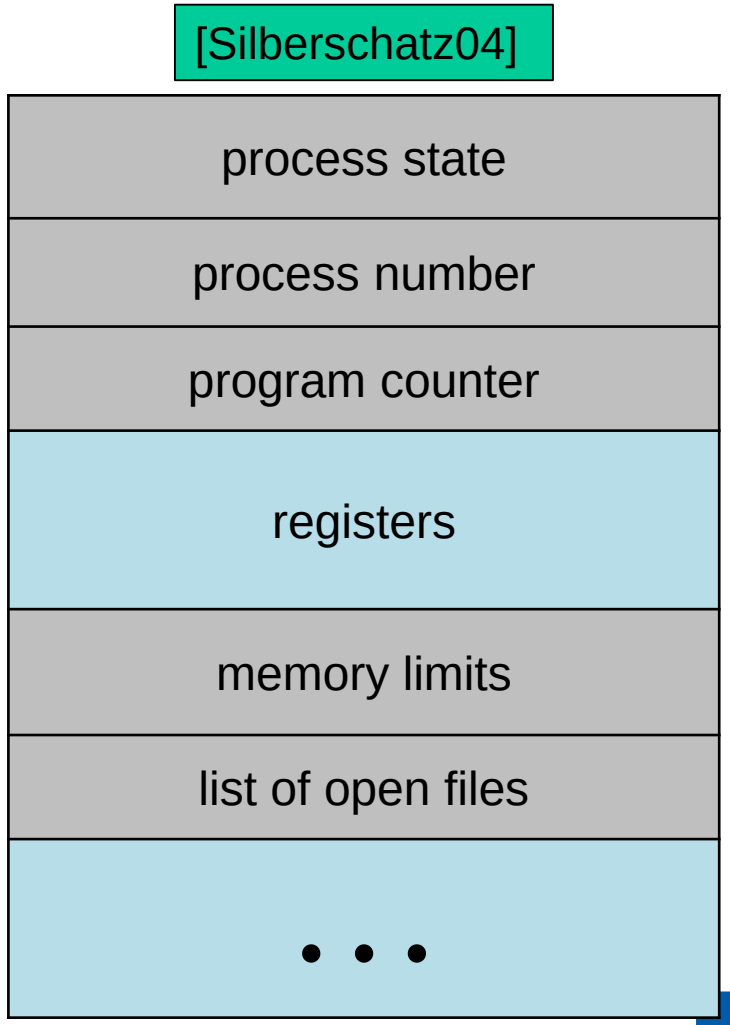


(Source: Wikipedia)

Process Control Block (PCB)

- Operating system must associate more information with each process

- This information is stored within a process control block (PCB)
  - Process state (more details follow)
  - Program counter
  - CPU registers
  - CPU scheduling information
  - Memory-management information
  - Accounting information
  - I/O status information



Context Switch

- Switching the processor from one process to another is commonly referred to as **context switch**
  - OS must save the state of the old process
  - OS must load the state for the new process

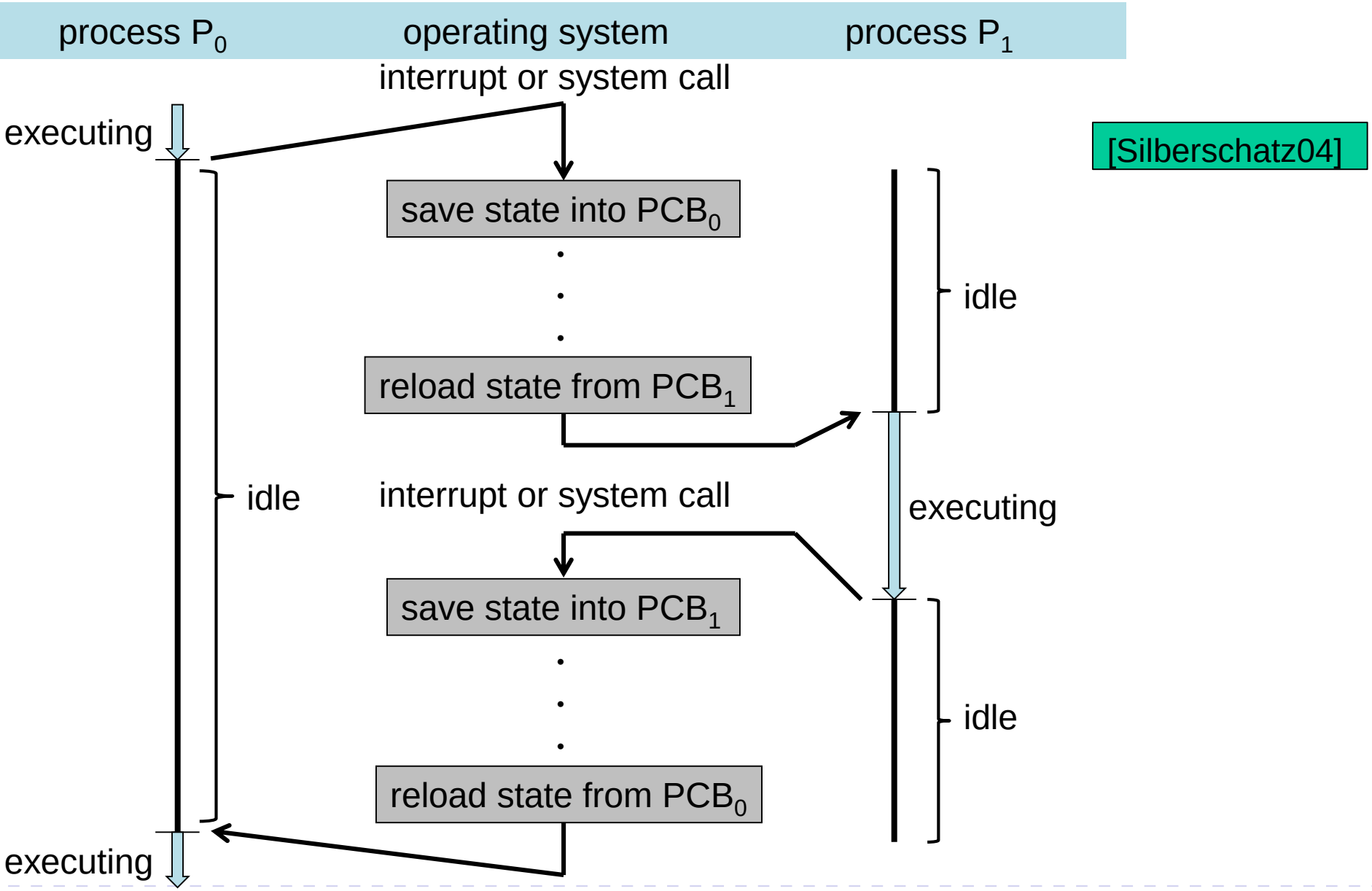


- **Context** of a process represented in the **PCB**

- For a context switch the **OS must be in control**
  - **System call** – process calls operating system procedure
  - **Interrupt handler** – device controller signals interrupt

- Context-switch time is overhead
  - The system does no useful work while switching
  - Time dependent on hardware support

Switch From Process to Process

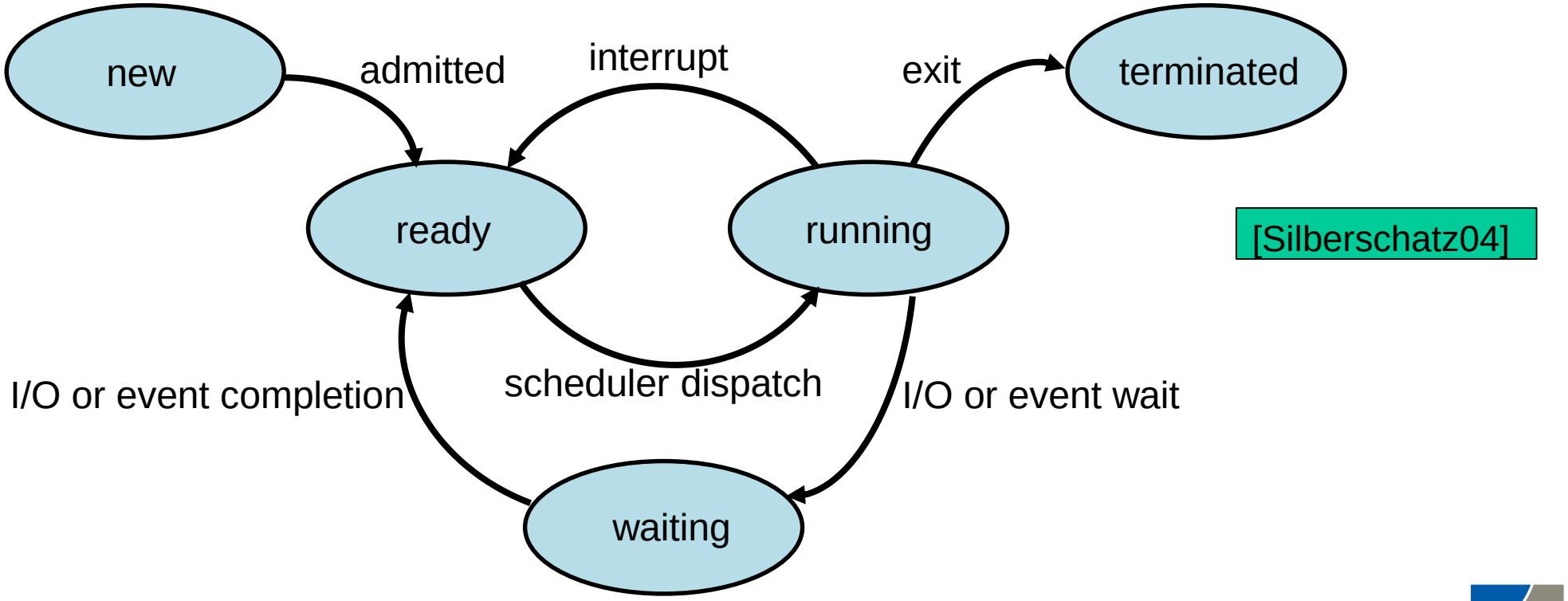


[Silberschatz04]

UNIVERSITÄT BONN

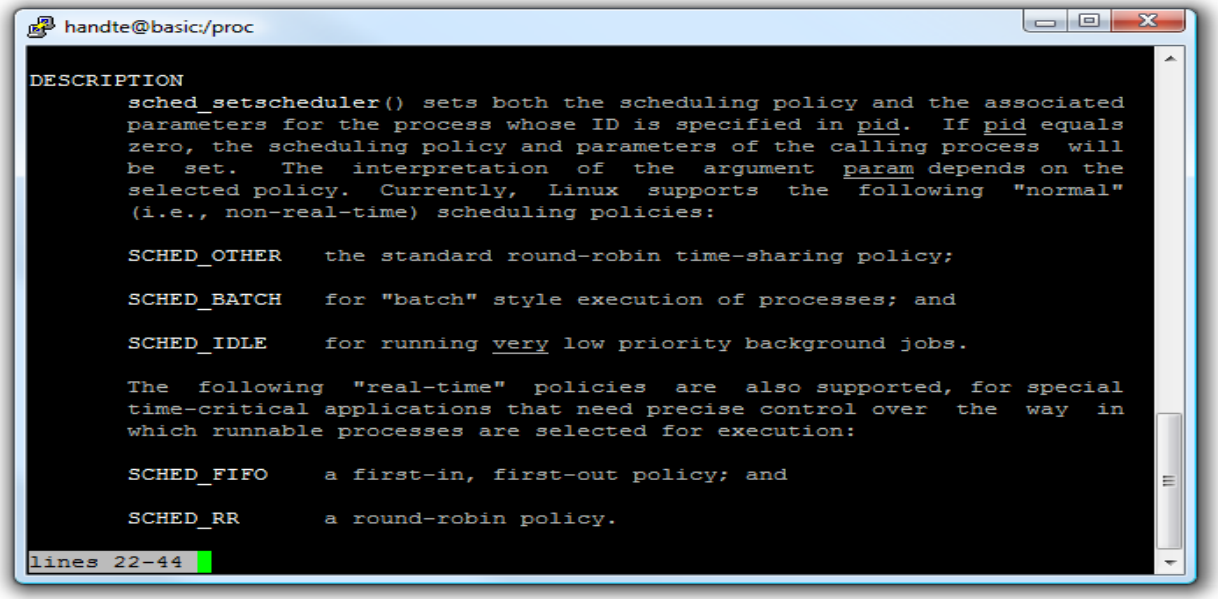
Process State

- As a process executes, it changes *state*
  - **new:** The process is being created
  - **running:** Instructions are being executed
  - **waiting:** The process is waiting for some event to occur
  - **ready:** The process is waiting to be assigned to a processor
  - **terminated:** The process has finished execution



Scheduling

see „man sched_setscheduler“



Linux Scheduling Policies

- **Before context switch OS decides which process to run (scheduling)**
  - Waiting processes cannot run but multiple processes might be ready
- **Scheduling goals may differ**
  - Batch system – high throughput
  - Interactive system – low latency
  - Embedded system – meet real-time deadlines

  => Different scheduling algorithms/policies to achieve different goals
- **Scheduler models**
  - Cooperative – Processes give up control by system call (yield)
  - Preemptive – Process may be stopped by system (e.g. upon timer interrupt)

Observations

- Code of a process is guaranteed to be executed **sequentially**
- Memory used by processes is **isolated** by OS
- **Feeling of parallel** execution is achieved by (fast) switching
- Depending on scheduler model, processes may be switched at any point in time during the execution
  - Execution duration of programs differs across multiple runs
  - Programmer cannot count on the switching time
  - Process implementation cannot count on real world timing of individual instructions
- Simplifies development of most types of applications but may complicate development of programs with real-time constraints

© 2026/2027 Dr. Matthias Frank, Dr. Matthias Wübbeling, Informatik 4

Process Creation in UNIX

- In most systems **processes are created dynamically**
  - Possible exceptions are embedded systems with fixed processes

- Process identified and **managed via a process identifier** (`pid`)

- Exemplary reasons for process creation
  - System initialization
  - User requests to execute a program
  - Batch job started by operating system
  - Running process starts another one

=> Technically, **all processes are created by another process**
  - Parent processes create children processes
  - Conceptually forming a tree of processes

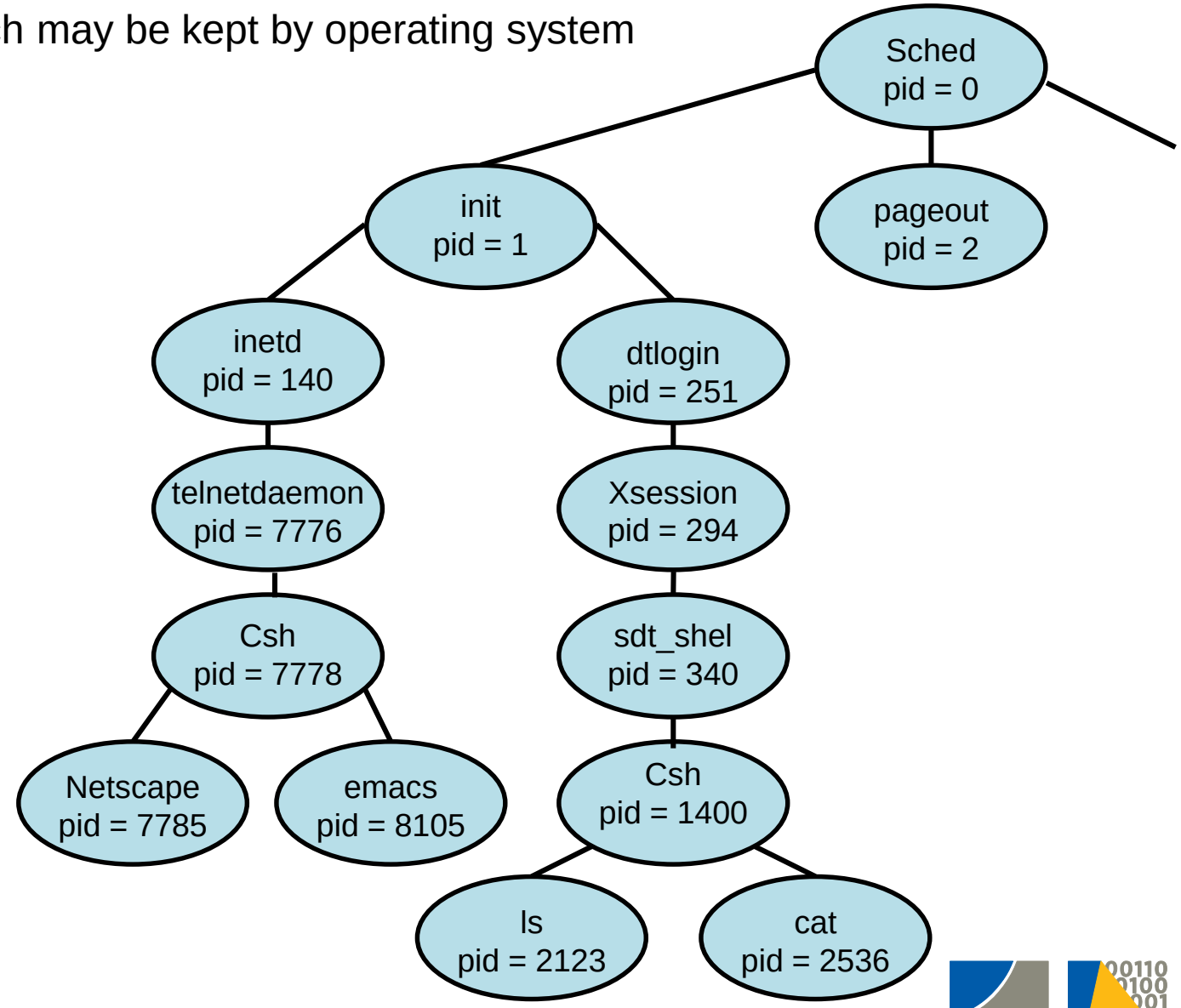
Process Hierarchies

[Silberschatz04]

- Parent/child relationship forms a **hierarchy** which may be kept by operating system

- **UNIX:**
  - Keeps the relationship
  - Referred to as "process group“
  - Child may get `pid` of parent (`ppid`)

- **Windows:**
  - No concept of hierarchy
  - All processes are created equal



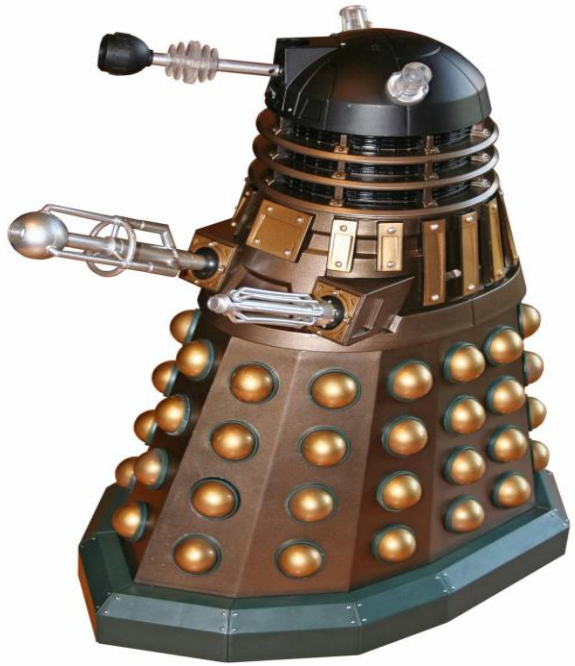
Exemplary process hierarchy on Solaris

Process Creation Options

- Operating system may support various options with respect to:
  - **Resource sharing**
    - Parent and children share all resources
    - Children share subset of parent's resources
    - Parent and child share no resources
  - **Execution**
    - Parent and children execute concurrently
    - Parent waits until children terminate
  - **Address space**
    - Child duplicate of parent
    - Child has a program loaded into it

Process Termination

- Processes may terminate for several reasons
  - **Process executes last statement and asks operating system to delete it** `(exit)`
    - Process indicates `success/failure` via return value
    - Output data transmitted from child to parent (via `wait`)
    - Process' resources are deallocated by operating system
  - **Parent process may terminate execution of child** `(abort)`
    - If child has exceeded allocated resources
    - If task assigned to child is no longer needed
    - If parent is exiting: some operating systems terminate all children upon parent termination (cascading termination)



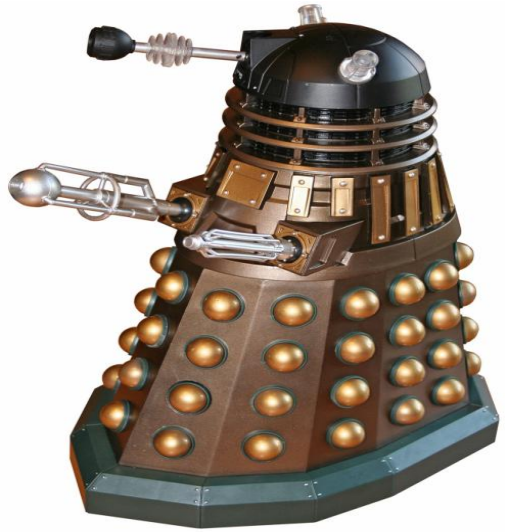
Process Termination (cont'd)

- Processes may also be terminated by other entities

- **Terminated involuntarily by operating system** `(fatal exit)`
  - Process exceeded memory limitations
  - Program code executes illegal instruction
  - Program code performs division by 0

  => Some conditions might be handled by process without exit

- **Terminated upon request by some other process**
  - E.g. terminated by a task manager program
  - Requires adequate privileges



Outline

- 2.1.1. Overview
  - Motivation
  - Concept
  - Implementation
  - Scheduling
  - Basic operations
- **2.1.2. Utilization**
  - Examples (Linux)
- 2.1.3. Summary

© 2026/2027 Dr. Matthias Frank, Dr. Matthias Wübbeling, Informatik 4

2.1.2. Utilization – Processes on Linux

- Status
- Creation
- Signals
- Termination
- Tools

Process Status in C

- Processes in Linux are arranged as a tree rooted at the **`init`** process

- Each process in Linux has
  - a process identifier (**`pid`**) to identify the process and
  - a parent process identifier (**`ppid`**) to identify its parent

- To obtain a process's **`pid`** and **`ppid`**, use the following system calls:

```c
#include <sys/types.h>
#include <unistd.h>

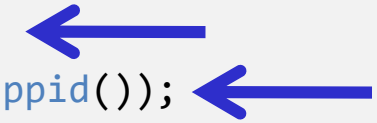
pid_t getpid(void);
pid_t getppid(void);
```

C Code Example (getpid(), getppid())

[Mitchell01]

- Exemplary program code that prints the process and parent process id:

```
int main ()
{
    printf("The process id is %d\n", (int) getpid());
    printf("The parent process id is %d\n", (int) getppid());
    return 0;
}
```



(Source: Advanced Linux Programming by CodeSourcery LLC, published by New Riders Publishing)

Anm.:
Alle Code-Ausschnitte mit Kennzeichen [Mitchell01] sind online verfügbar, s. Folie 3

Process Creation



- Two classic ways to create a process in Linux:
  - Using the `system()` function to execute a command
  - Using `fork()` and `exec()` system calls

- Using the `system()` function to execute a command
  - `system()` creates a sub process to run the standard shell (`/bin/sh`), which then executes the command
  - `system()` returns the exit status of the shell command
    - 127, if the shell fails to run
    - -1, other errors

Code Example (system)

[Mitchell01]

- Exemplary program code that invokes “`ls -l`“ on the root directory

```
#include <stdlib.h>

int main()
{
    int return_value;
    return_value = system(“ls -1 /”);
    return return_value;
}
```

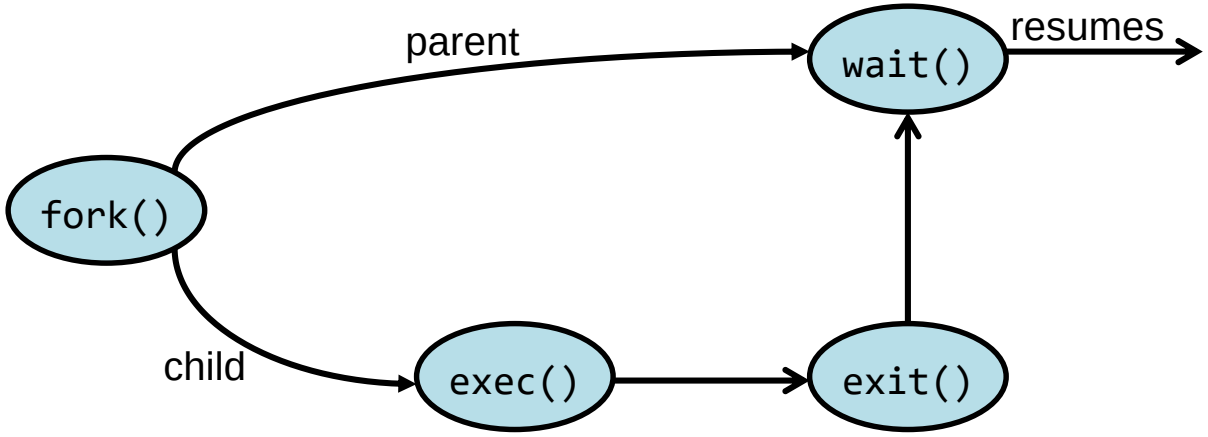


(Source: Advanced Linux Programming by CodeSourcery LLC, published by New Riders Publishing)

Process Creation (2)

- Using `fork()` and `exec()` system calls
  - `fork()` creates a child process with the returned values
    - child process's pid in the parent process, and
    - zero in the child process
    - In the past: child process basically a clone of parent
    - Nowadays: Copy-on-write for performance
- `exec()` executes the sub program
  - current process is replaced with the new process
- `exit()` signals normal completion
- `abort()` signals abnormal completion
- `wait()` awaits the process completion

[Silberschatz04]



Process Creation (3)

- Several variants of `exec()`
  - List of arguments specified as null-terminated strings, last argument must be null pointer
    - `int execl(const char *path, const char *arg, ...);`
    - `int execlp(const char *file, const char *arg, ...);`
    - `int execle(const char *path, const char *arg, ..., char * const envp[]);`
- List of arguments specified as array of null-terminated strings
  - `int execv(const char *path, char *const argv[]);`
  - `int execvp(const char *file, char *const argv[]);`
- `execlp` and `execvp` will search for executable in `PATH` if leading / not specified
- `execle` enables specification of environment

„l“ = list, „v“ = vector (either one to be used)

„e“ = environment (to be spec.)

„p“ = search in PATH

Code Example (fork, exec, exit)

- Exemplary program code that invokes “ls“ and waits for child

[Silberschatz04]

```
int main()
{
    pid_t pid;

    /* fork another process */
    pid = fork();
    if (pid < 0) { /* error ocurred*/
    fprintf(stderr, “Fork Failed”);
    exit(-1);
    }
    else if (pid == 0) { /* child process */
    execlp(“/bin/ls”, “ls”, NULL);
    }
    else { /* parent process */
    /* parent will wait for the child to complete */
    wait(NULL);
    printf(“Child Complete”);
    exit(0);
    }
}
```

C Code Example (fork, exec, abort)

- Exemplary program code that invokes “ls -l“ on the root directory

[Mitchell01]

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>

/*  Spawn a child process running a new program. PROGRAM is the name of the
    program to run; the path will be searched for this program. ARG_LIST is
    a NULL-terminated list of character strings to be passed as the
    program's argument list. Returns the process ID of the spawned process.
    */

int spawn (char *program, char **arg_list)
{
    pid_t child_pid;

    /* Duplicate this process */
    child_pid = fork();
    if (child_pid != 0)
    /* This is the parent process. */
    return child_pid;
    else {
    /* Now execute PROGRAM, searching for it in the path. */
    execvp(program, arg_list);
    /* The execvp function returns only if an error occurs. */
    fprintf(stderr, “an error occurred in execvp\n”);
    abort();
    }
}
```

```c
int main ()
{
    /* The argument list to pass to the “ls” command. */
    char *arg_list[] = {
    “ls”,  /* argv[0], the name of the program. */
    “-l”,
    “/”,
    NULL   /* The argument list must end with a NULL. */
    };

    /* Spawn a child process running the “ls” command.
    Ignore returned child process ID. */
    spawn(“ls”, arg_list);

    printf(“done with main program\n”);

    return 0;
}
```

```
int spawn (char* program, char** arg_list)
```

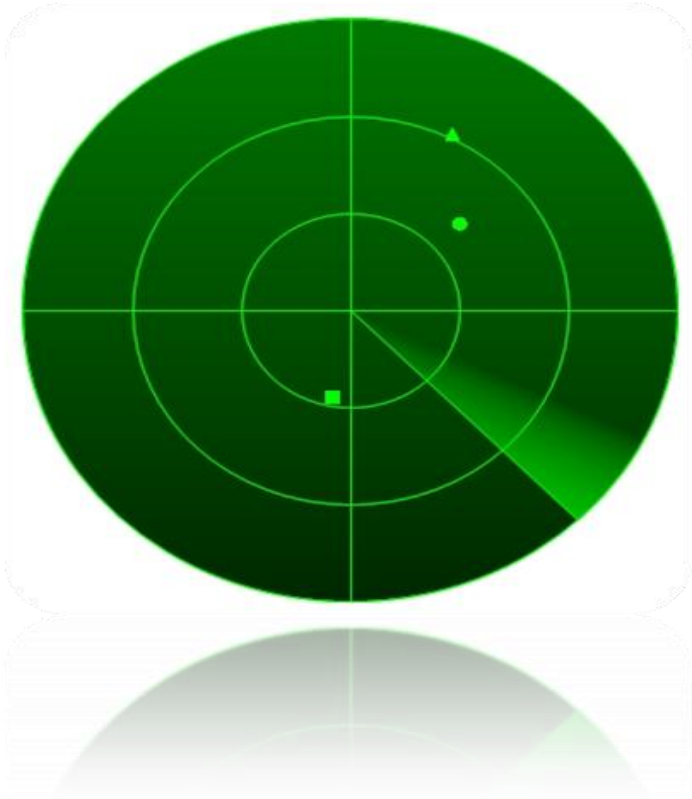
(Source: Advanced Linux Programming by CodeSourcery LLC, published by New Riders Publishing)

Signals

- Signals are:
  - mechanisms from communicating and manipulating processes
  - special messages sent to a process
  - asynchronous

- Normally, when a process receives a signal, it
  - first stops the current execution,
  - handles the signal, and then
  - resumes the original execution.

- Each signal type is specified by a signal number
- For most signal types, a program can respond to the signal by registering a signal-handler function



Signal Types

- Linux defines several types of signals and default actions
  - Some examples are (according to POSIX.1-1990 standard)

Signal	Value	Action	Comment
SIGHUP	1	Term	Death of controlling process
SIGINT	2	Term	Interrupt from Keyboard
SIGQUIT	3	Core	Quit from Keyboard
SIGILL	4	Core	Illegal Instruction
SIGABRT	6	Core	Abort signal from abort
SIGFPE	8	Core	Floating point exception
SIGKILL	9	Term	Kill signal
SIGSEGV	11	Core	Invalid memory reference
SIGPIPE	13	Term	Broken pipe: write to pipe with no readers
SIGALRM	14	Term	Timer signal from alarm
SIGTERM	15	Term	Termination signal
SIGUSR1	30, 10, 16	Term	User-defined signal 1
SIGUSR2	31, 12, 17	Term	User-defined signal 2
SIGCHLD	20, 17, 18	Ign	Child stopped or terminated
SIGCONT	19, 18, 25	Cont	Continue if stopped
SIGSTOP	17, 19, 23	Stop	Stop process
SIGTSTP	18, 20, 24	Stop	Stop typed at tty
SIGTTIN	21, 21, 26	Stop	tty input for background process
SIGTTOU	22, 22, 27	Stop	tty output for background process

The signals SIGKILL and SIGSTOP cannot be caught, blocked, or ignored.

Code Example (signal handler)

[Mitchell01]

- Exemplary program code that registers handler to count the number of received SIGUSR1 signals

```
#include <signal.h>
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <unistd.h>

sig_atomic_t sigusrl_count = 0;
void handler(int signal_number)
{
    ++sigusrl_count;
}

int main()
{
    struct sigaction sa;
    memset(&sa, 0, sizeof(sa));
    sa.sa_handler = &handler;
    sigaction(SIGUSR1, &sa, NULL);

    /* Do some lengthy stuff here. */
    /* ... */

    printf("SIGUSR1 was raised %d times\n", sigusr1_count);
    return 0;
}
```

(ATOMIC)

(Source: Advanced Linux Programming by CodeSourcery LLC, published by New Riders Publishing)

Process Termination

- Wait for process termination in C
  - Parent process may want to retrieve the termination status of its child processes
  - `wait()` function is used to obtain the exit code (integer)
  - `WIFEXITED` macro is used to determine a child process's exit status
  - `WEXITSTATUS` macro can be used to extract the exit code

Code Example (wait)

- Exemplary program code that uses macros to wait for child

[Mitchell01]

```
int main()
{
    int child_status;
    /* The argument list to pass to the “ls” command. */
    char *arg_list[] = {
    “ls”,                    /* argv[0], the name of the program. */
    “-l”,
    “/”,
    NULL           /* The argument list must end with a NULL. */
    };

    /* Spawn a child process running the “ls” command. Ignore the returned child process ID. */
    spawn(“ls”, arg_list);
    /* Wait for the child process to complete. */
    wait(&child_status);
    if (WIFEXITED(child_status))
    printf(“The child process exited normally, with exit code %d\n”,
    WEXITSTATUS(child_status));
    else
    printf(“The child process exited abnormally\n”);
    return 0;
}
```

(Source: Advanced Linux Programming by CodeSourcery LLC, published by New Riders Publishing)

Zombies

[Mitchell01]

- A zombie process is a child process that has terminated but its parent process has not called wait()
- Exemplary code for creating a zombie process

```
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>

int main()
{
    pid_t child_pid;

    /* Create a child process. */
    child_pid = fork();
    if(child_pid > 0) {
    /* This is the parent process. Sleep for a minute. */
    sleep(60);
    }
    else {
    /* This is the child process. Exit immediately. */
    exit(0);
    }
    return 0;
}
```

(Source: Advanced Linux Programming by CodeSourcery LLC, published by New Riders Publishing)

Zombies (2)

- If parent process calls **`wait()`** before the child process terminates,
  - the parent process blocks until the child process terminates

- If child process has terminated,
  - it terminates with its exit status code if its parent process has called **`wait()`**
  - it becomes a zombie process if its parent process has not called **`wait()`**
  - it is cleaned up by the **`init`** process if its parent process exits without calling **`wait()`**



Asynchronous Cleanup

- A parent process may want to continue to work without being blocked by **`wait()`**
- One approach is to periodically call **`wait3()`** or **`wait4()`** with non-blocking mode
- Another approach is to use the signal **SIGCHLD**
  - Parent process is notified when a child process terminates
  - Parent specifies a signal handler, which stores the child process's termination status, and then cleans up the child

© 2026/2027 Dr. Matthias Frank, Dr. Matthias Wübbeling, Informatik 4

Code Example (SIGCHLD)

[Mitchell01]

- Exemplary program code that registers handler to clean up child

```c
#include <signal.h>
#include <string.h>
#include <sys/types.h>
#include <sys/wait.h>

sig_atomic_t child_exit_status;

void clean_up_child_process(int signal_number)
{
    /* Clean up the child process. */
    int status;
    wait(&status);
    /* Store its exit status in a global variable. */
    child_exit_status = status;
}

int main()
{
    /* Handle SIGCHLD by calling clean_up_child_process. */
    struct sigaction sigchld_action;
    memset(&sigchld_action, 0, sizeof(sigchld_action));
    sigchld_action.sa_handler = &clean_up_child_process;
    sigaction(SIGCHLD, &sigchld_action, NULL);

    /* Now do things, including forking a child process. */
    /* ... */

    return 0;
}
```

(Source: Advanced Linux Programming by CodeSourcery LLC, published by New Riders Publishing)

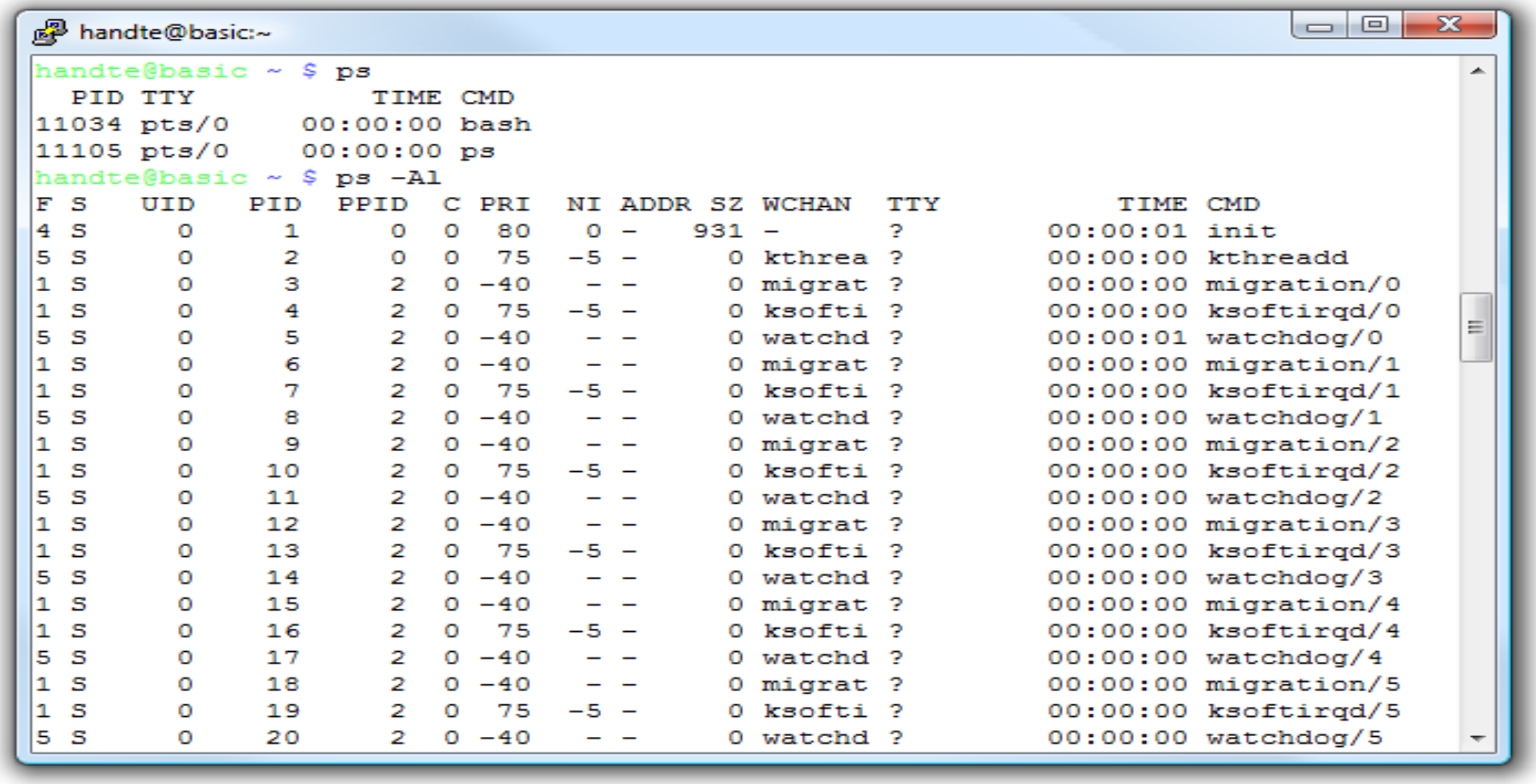
Process Tools in Linux

- **View running processes**
  - `ps, pstree, top`
- **Send signals to processes**
  - `kill`
- **Modify process execution**
  - `nice / renice`
  - `& / <ctrl+z> / fg / bg`

© 2026/2027 Dr. Matthias Frank, Dr. Matthias Wübbeling, Informatik 4

Ps

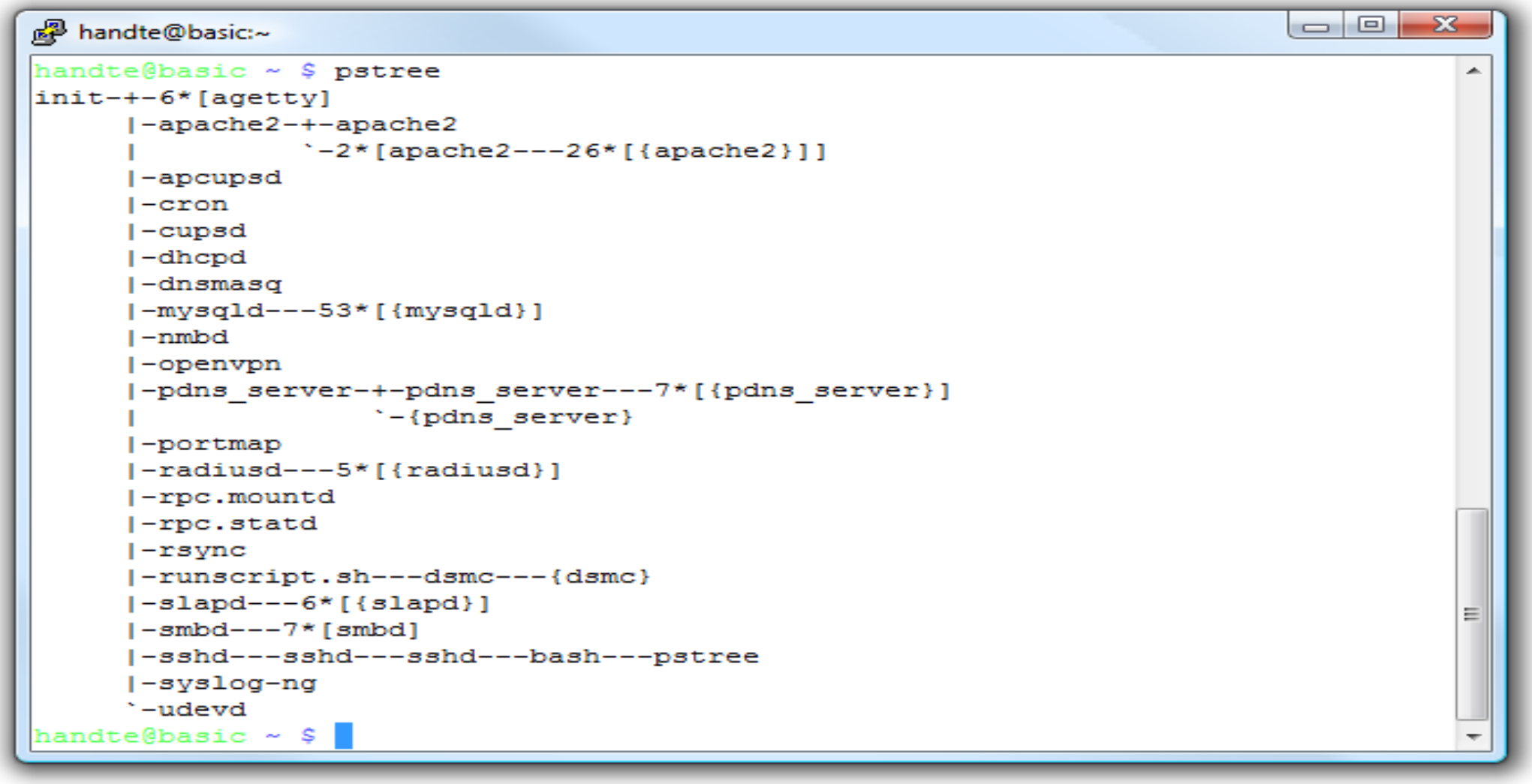
- `ps` – report a snapshot of the current processes



© 2026/2027 Dr. Matthias Frank, Dr. Matthias Wübbeling, Informatik 4

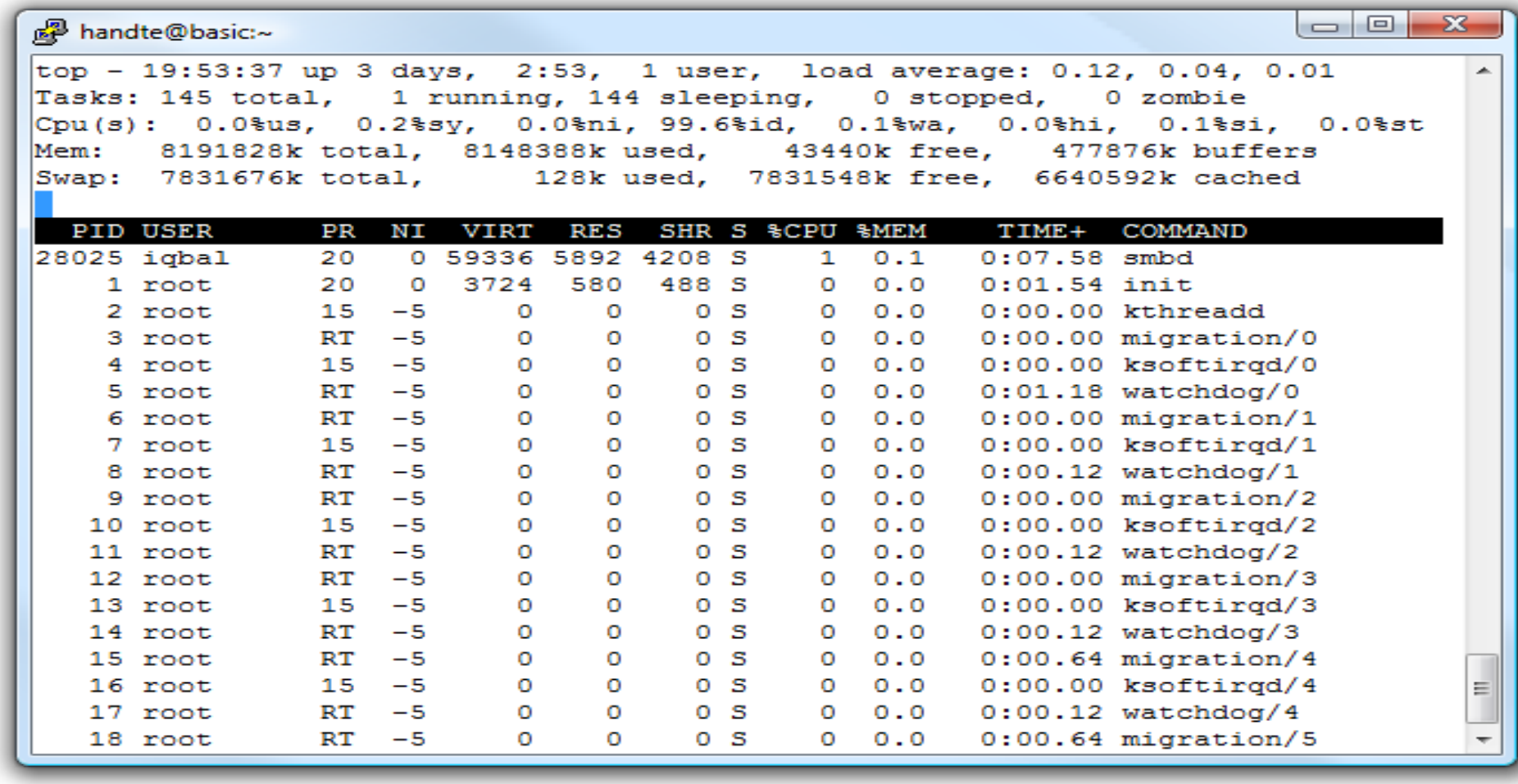
Pstree

- `pstree` – displays a tree of running processes



Top

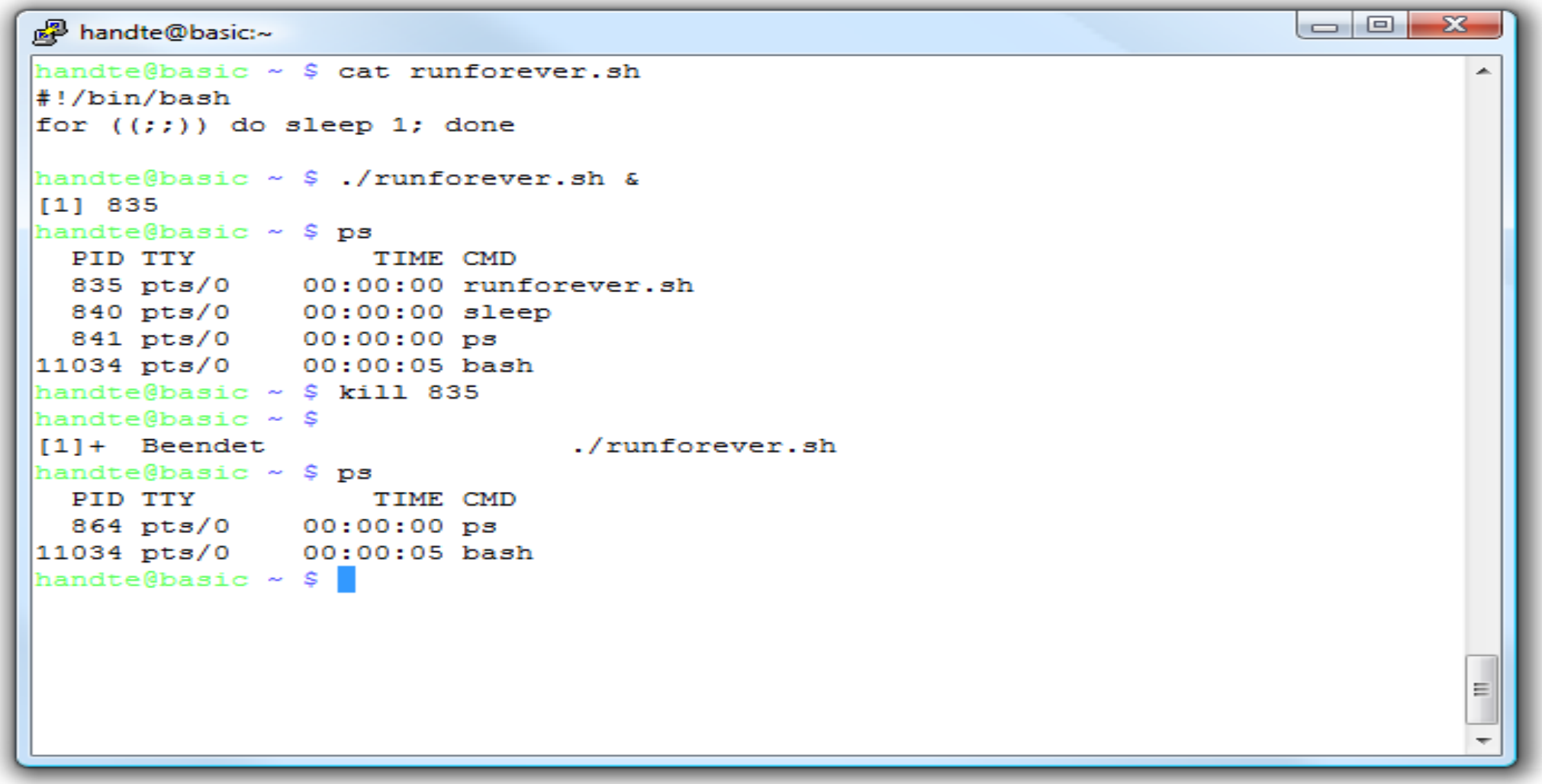
- `top` – displays processes (table can be rearranged interactively)
- `htop` – advanced top



© 2026/2027 Dr. Matthias Frank, Dr. Matthias Wübbeling, Informatik 4

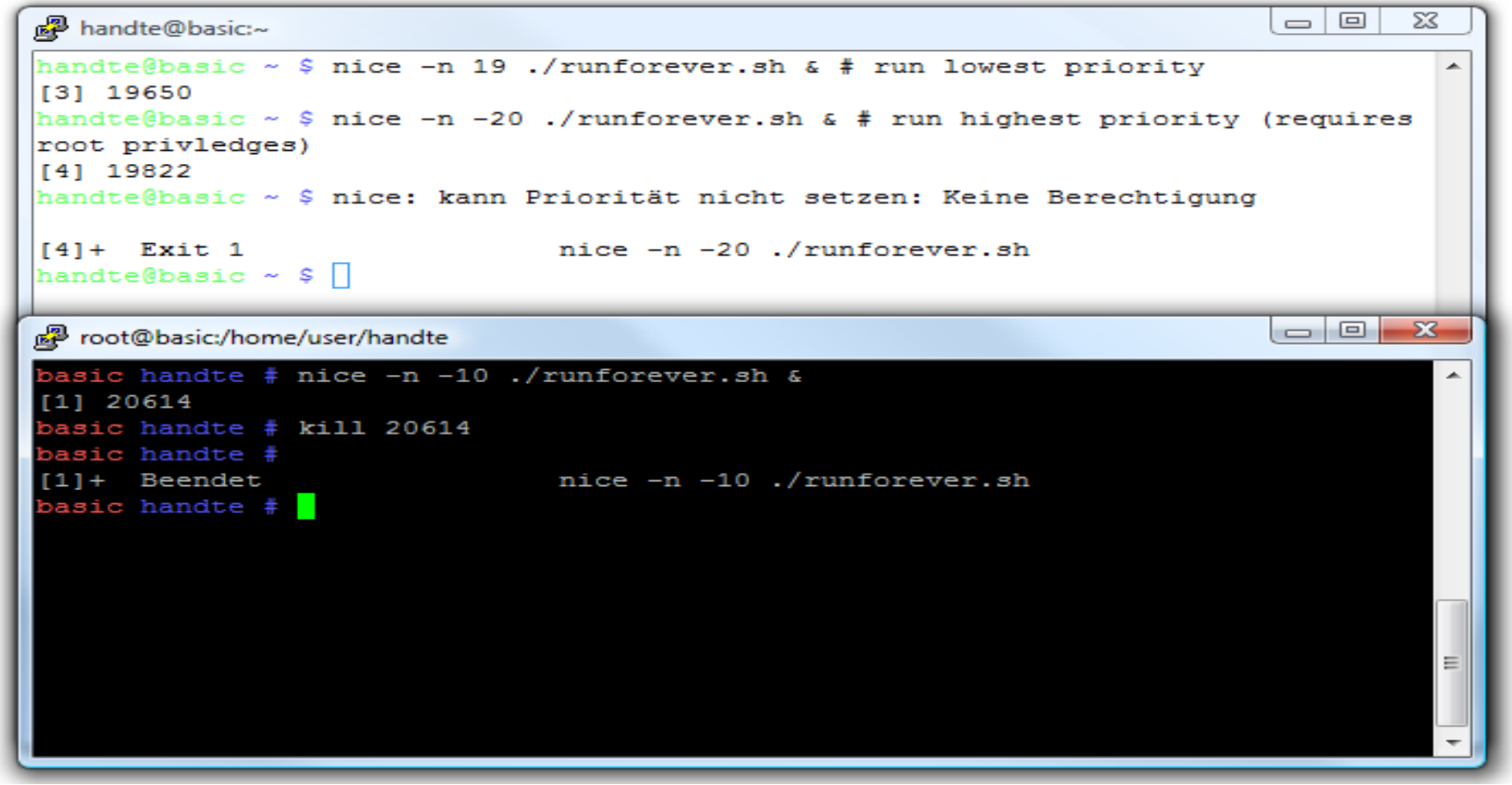
Kill

- `kill` – send signals to a process (defaults to SIGTERM)



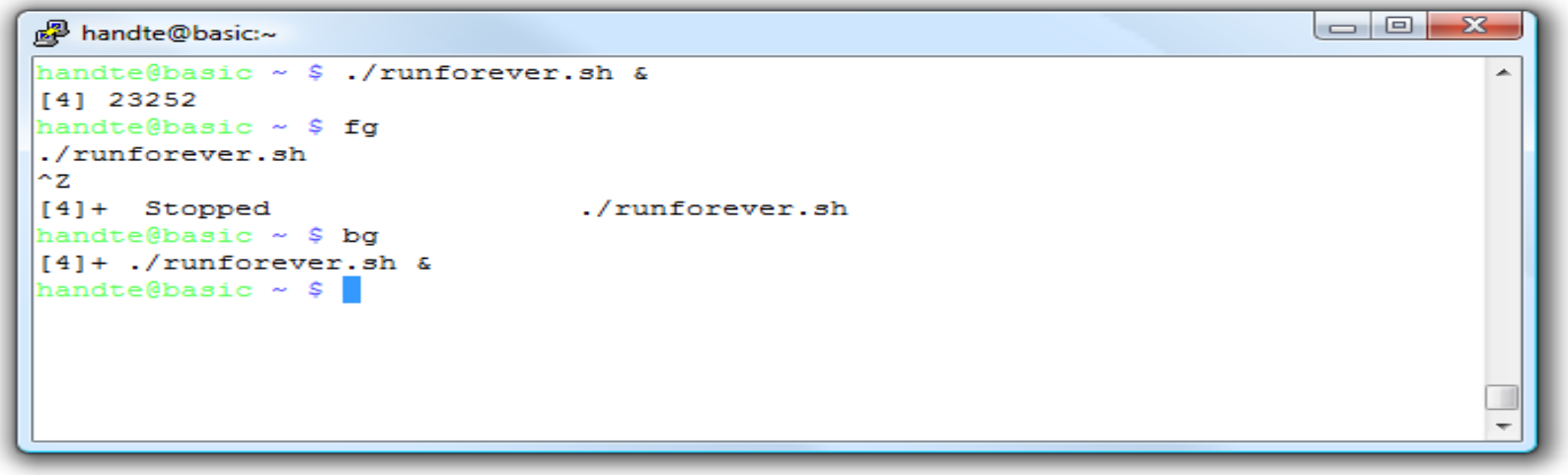
Nice / Renice

- `nice` – change scheduling priority upon startup
- `renice` – change scheduling priority of running process



& / <Ctrl+Z> / Fg / Bg

- & – start process in background
- `<ctrl+z>` – stop process in foreground
- `bg` – move process to background
- `fg` - move process to foreground



Outline

- 2.1.1. Overview
  - Motivation
  - Concept
  - Implementation
  - Scheduling
  - Basic operations
- 2.1.2. Utilization
  - Examples (Linux)
- **2.1.3. Summary**

2.1.3. Summary

- Processes are an abstraction of an executing program instance
  - Simplifies program development and processor usage
  - In C, programmers refrain from assumptions on the execution of
  - Multiple processes (switched in and out arbitrarily)
  - Timing of instructions within a process (blocked / switched out)
  - The OS maps processes on available processors
  - Scheduling can be optimized for system purposes

- Most modern operating systems
  - provide calls for process creation, manipulation and termination
  - hide the low-level details from the programmer
  - Creating space for data (registers, program counter, etc.)
  - Transparent storing and restoring of process context

Limitations

- Process creation is not a light-weight undertaking

- Parallelism/Concurrency may be required even within single application
  - E.g. a web server processing multiple requests simultaneously

- Memory isolation provided by processes is not always necessary and may be more complicated to deal with
  - E.g. requires explicit data sharing between different processes

… IPC (inter process communication) in subsection 2.3.

- In many cases, threads can provide a more light-weight alternative …
  … but more to come in next subsection

… threads in subsection 2.2.